VarTable

VarTable is a package to make creation of table of signs easier.
This package is built on Cetz.
(version : 0.2.1)

Table of Contents

1 - Introduction 3
2 - General description 4
tabvar 4
Parameters 4
variable 4
label 4
domain 5
contents 5
table-style 5
nocadre 5
arrow-mark 5
arrow-style 5
line-0 6
undefined-bar-alt 6
line-style 6
hatching-style 6
first-column-width 6
first-line-height 6
element-distance 7
values 7
fill-color 7
add 7
3 - Content parameter 8
3.1 - Signs format 8
3.1.1 - A classical array for signs 8
3.1.2 - A customed separation bar 9
3.1.2.1 - The style of the bar 9
3.1.2.2 - The type of the bar 9
3.1.3 - Ignoring a value 11
3.1.4 - Hatching for an undefined interval 12
3.2 - Varitions format 14
3.2.1 - A classical array for sub-tables of variations 14
3.2.2 - Undefined values 14
3.2.3 - Ignoring a value 16
3.3 - Hatching for an undefined interval 16
4 - Resizing the table 19
4.1 - First line and column 19
4.2 - Resizing the spacing between elements 19
4.3 - Resizing the height of sub-tables 20
5 - Customing the hatching 20
6 - Adding values into sub-tables of variations 24

7 - Add what you want with `add` 26

1 - Introduction

This package was built to make creation of tables of signs easier. In order to do so, the package provides the « `tabvar` » function, the arguments of which are described in this documentation.

If you find a bug, thank you to warn me via my GitHub.

P.S : I know my english is not the best, so if you find this documentation too hard to read and you have a few time to lose, you are most welcomed to improve it.

Aknowledgments : I wanted to thank supersurviveur and dododu74, for their help in the beginning of the project, (especially for the correction of the first documentations).
As well as Akilon27, without whom, the tables would not be so customisable.

2 - General description

tabvar

Return a variation table

Parameters

```
tabvar(
  variable: content,
  label: array,
  domain: array,
  contents: array,
  table-style: style,
  nocadre: bool,
  arrow-mark: mark,
  arrow-style: style,
  line-0: bool,
  undefined-bar-alt,
  line-style: style,
  hatching-style: tiling,
  first-column-width: length,
  first-line-height: length,
  element-distance: length,
  values: array,
  fill-color: color,
  add: content
)
```

variable `content`

variable is a content block which contains the table's variable name (like x or t)

Default: `$ x $`

label `array`

`label` is an array which contain arrays of 2 arguments that contains in first position the line's label and in second position, if the line is a variation table or a sign table with this following keys : « v » for variation and « s » for sign
Example : for a variation table of a function f, you should write :

```
init: (
  variable: $x$,
  label: (
    ([sign of $f$], "Sign"), // the first line is a sign table
    ([variation of $f$], "Variation") // the second line is a variation table
  )
)
```

Default: `()`

domain `array`

values taken by the variable
for example if your funtions changes sign or reaches a max/min for $x \in \{0, 1, 2, 3\}$
you should write this :

```
domain: ($0$, $1$, $2$, $3$)
```

Default: `()`

contents `array`

the content of the table
see below for more details

Default: `((),)`

table-style `style`

Optional
The style of the table,
the style type is defined by Cetz, so I invite you to have a look at the Cetz manual.
Caution : if you haven't entered the mark symbol as none, all lines in the table will have an arrowhead.

Default: `(stroke: 1pt + black, mark: (symbol: none))`

nocadre `bool`

to hide the external cardre

Default: `false`

arrow-mark `mark`

the style of the arrowhead, the type of which is defined by Cetz

Default: `(end: "straight")`

arrow-style `style`

Optional
the style of the arrow, as for the `table-style` parameter
Caution : the `mark` section is overwrite by the `arrow-mark`

Default: `(stroke: black + 1pt)`

line-0 `bool`

Optional
if you want to change the default bar sign to a bar with a 0

Default: `false`

undefined-bar-alt

Default: `false`

line-style `style`

Optional
if you want to change the style of all separator lines between signs

Warning: this will only change the default lines, the `||`, `|` or `0` lines will not be changed.

Default: `(stroke: black + 1pt)`

hatching-style `tiling`

Optional
The style of the hatching in the sign and variation table

Default: `tiling(size: (30pt, 30pt))[`

```
  #place(line(start: (0%, 100%), end: (100%, 0%), stroke: 2pt))
  #place(line(start: (-100%, 100%), end: (100%, -100%), stroke: 2pt))
  #place(line(start: (0%, 200%), end: (200%, 0%), stroke: 2pt))
]
```

first-column-width `length`

Optional
change the width of the first column

Default: `none`

first-line-height `length`

Optional
change the height of the first line

Default: `none`

element-distance `length`

Optional
change the distance betwen two elements

Default: `none`

values `array`

Optional
To add values betwen to pre-defined values

Default: `((),)`

fill-color `color`

Optional
The fill color for values (should be the same as the background of the table) default : white

Default: `white`

add `content`

Optional
To add more stuff with Cetz

Default: `()`

3 - Content parameter

The content parameter is an array with an element per line (per label).

Each element itself is an array with an element for each column, with a different format for signs and variations that will be described more below.

3.1 - Signs format

It has to be placed at the same index in the `contents` array as a label possessing a `"s"` string, it means the line has to be considered as a table of signs.

Additionally, it must contain as many elements as the domain minus one (one per interval), plus an optionnal argument if the last argument is undefined.

Each element must be in one of the following forms:

- `()` - Empty: to extend the last sign from the left to the right on every interval marked empty
- `body` - The basic case, composed of the body type from typst, like `$ + $` or `$ - $`
- `(style of the bar, body)` - to specify a particular type for the bar **before** he sin, this style an be: `"|"` simple bar, `"||"` double bar or `"0"` for a bar with zero in the middle. It can also be any content (`[…]`, `$…$`, `box(…)`): it is placed in the middle of the bar, like the zero — for instance an empty box to fill in, `(box(width: 0.8em, height: 0.8em, fill: white, stroke: (dash: "dotted")), $-$)`, please note that the table will not adapt to this insertion.

NB: The `line-0` parameter changes the defaut bar for the bar with the zero `"0"`.

You can also ad at the `"||"` string at the end, to add a double bar at the end of the table.

3.1.1 - A classical array for signs

A classical array for signs:

```
#tabvar(
  init: (
    variable: $t$,
    label: (([sign], "s"),),
  ),
  domain: ($2$, $4$, $6$, $8$),
  contents: (($+$, $-$, $ + $),),
)
```

t	2	4	6	8
sign	+	−	+	

A more complex example:

```
  #tabvar(
    variable: $t$,
    label: (
      ([sign 1], "s"),
      ([sign 2], "s"),
    ),
    domain: ($ 2 $, $4$, $6$, $8$),
    contents: (
      ("Hello world !", $-$, $ 3 / 2 $),
      ($1/3/9/27$, $+$, "Goodbye
world !"),
    ),
  )
```

t	2	4	6	8
sign 1	Hello world !	$-$	$\frac{3}{2}$	
sign 2	$\frac{\frac{1}{3}}{\frac{9}{27}}$	$+$	Goodbye world !	

Note : In the second example, the table is compressed with a scale function.

3.1.2 - A customed separation bar

3.1.2.1 - The style of the bar

You can modify the style of the bar.

The style of the bar is a dictionary, of `"style"` type defined by Cetz.
To make it simple, if you want to change the stroke of the bars only, you juste need to put `(stroke: your stroke)`.
For more complexe uses see Cetz textbook.

Example :

```
  #tabvar(
    line-style: (
      stroke: (paint: red, dash: "dashed")
    ),
    variable: $t$,
    label: (([sign], "s"),),
    domain: ($2$, $4$, $6$,),
    contents: (
      ($+$, $-$),
    ),
  )
```

t	2	4	6
sign	$+$		$-$

3.1.2.2 - The type of the bar

For all signs exept the first instead of putting directly a sign you can put a couple, the first element of which defines the type of the bar placed before the sign.
There are 3 different types of bars :

- `"|"` : a simple bar
- `0` : a bar with a zero at the center
- `||` : a double bar, for undefined values

Example

```
#tabvar(
  variable: $t$,
  label: (([sign], "s"),),
  domain: ($2$, $4$, $6$, $8$, $10$),
  contents: (
    (
      $+$,
      ("|", $-$),
      ("0", $-$),
      ("||", $+$)
    ),
  ),
)
```

t	2	4	6	8	10
sign	$+$	$-$	0	$-$	$+$

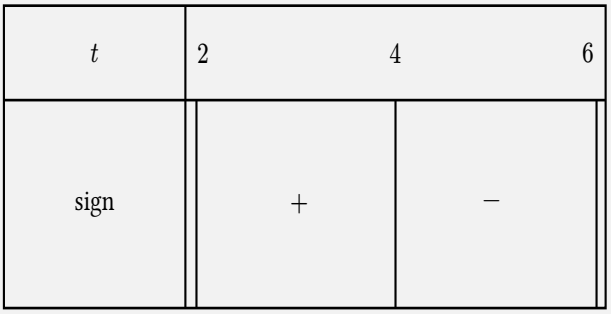
If you want a double bar before the first sign,you can use a couple with a `"||"` first element instead of the first sign ; for a double bar in the end, add a `"||"` string at the end of the array, all other types of bars are also supported, notably the insertion of any content in the middle of the bar..

Example :

```
#tabvar(

  variable: $t$,
  label: (([sign], "s"),),

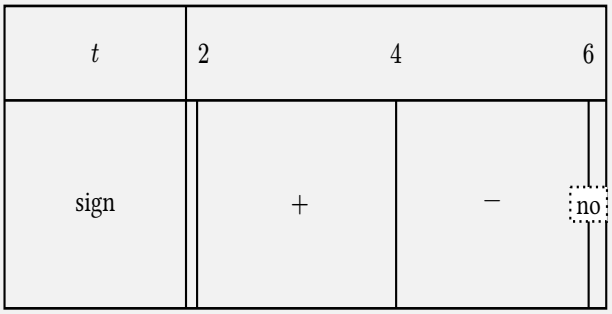
  domain: ($2$, $4$, $6$),
  contents: (
    (
      ("||", $+$),
      $-$,
      "||"
    ),
  ),
)
```




```
#tabvar(

  variable: $t$,
  label: (([signe], "s"),),

  domain: ($2$, $4$, $6$),
  contents: (
    (
      ("||", $+$),
      $-$,
      box(
        fill: white,
        stroke: (dash: "dotted"),
        inset: 1mm
        )[
          non
        ]
    ),
  ),
)
```



3.1.3 - Ignoring a value

If your table of signs is composed of more than one subtable, you might be tempted to put the same sign for several values of the domain.
This is quite simple, instead of putting directly a sign, just put an empty couple `()`.

Example :

```
#tabvar(
  line-0: true,
  variable: $t$,
  label: (
    ([sign], "s"),
  ),
  domain: ($2$, $4$, $6$, $8$),
  contents: (
    ($+$, (), $-$),
  ),
)
```

t	2	4	6	8
sign		+	0	−

3.1.4 - Hatching for an undefined interval

Your function might not be defined on one or several intervals unfortunalty contained inside the domain of the table of signs, by convention we hatch said zone.
Since the signs refer to the intervals of domain, the result is a relatively simple syntax to use, we can separate 4 cases :

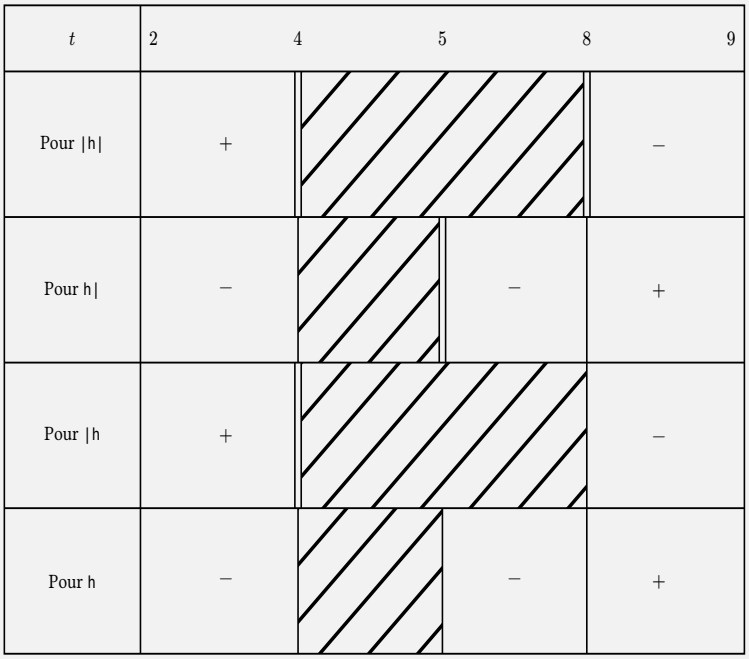
- the first and most usual case, when the two bounds of an undefined interval are undefined as well, put, where you would have put the sign (or any other element), the element : `"|h|"`
- the second case, relatively usual too, is when the two bounds are defined, you remove the « | » of previous case, i.e. you are left with `"h"`
- the two other cases are when only one of the bounds is defined, as you may have understood you put a « | » on the side in which the element is defined, meaning : for a defined value on the left `"h|"`; for a defined value on the right `"|h"`.

Note : You may have understood that « | » symbolises the double undefined bars, as well as « h » represent the « h » of hatch, therefore it is natural to put or not the bars if needed.

To extend the hatch to more than one interval of the domain, you anly need to pass next element with the same notation `()`.

Example :

```
  #tabvar(
    variable: $t$,
    label: (
      ([Pour `|h|`], "s"),
      ([Pour `h|`], "s"),
      ([Pour `|h`], "s"),
      ([Pour `h`], "s"),
    ),
    domain: ($ 2 $, $4$, $5$, $8$,
$9$),
    contents: (
      ($+$, "|h|", (), $-$),
      ($-$, "h|", $-$, $+$),
      ($+$, "|h", (), $-$),
      ($-$, "h", $-$, $+$),
    ),
  )
```



3.2 - Varitions format

It must be placed at the same index as a label of `"v"` string in the `contents` array, it shows that the line must be considered as a table of signs.

Additionally there have to be as many elements as in the domain, if not the program will return an error.

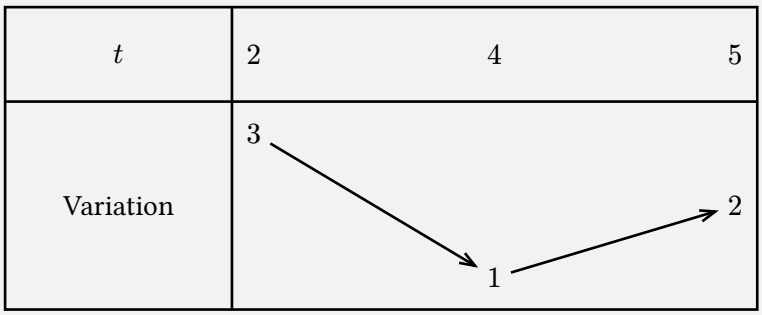
Each of the composing elements must be of one of the following forms :

- `()` - Empty : To extend the arrow to next element
- `(position, body)` - Classic case, composed of the element's position (top, center, bottom), and of the body
- `(pos1, pos2, "||", body1, body2)` - Le cas où l'élément est non défini
- `(pos, "||", body)` - A condensed writing of previous form
- `"h"` ou `"|h"` - The beginning tag of a hatched zone
- `"H"` ou `"H|"` - The ending tag of a hatched zone

3.2.1 - A classical array for sub-tables of variations

An array for the sub-tables of variations must contain at least two elements, being the position and the element itself.
the position can be either : `top`, `center` or `bottom`, but cannot be of another « alignement » type.

```
#tabvar(
  variable: $t$,
  label: (
    ([Variation], "v"),
  ),
  domain: ($ 2 $, $4$, $5$),
  contents: (
    (
      (top, $3$),
      (bottom, $1$),
      (center, $2$),
    ),
  ),
)
```



3.2.2 - Undefined values

If your function is undefined for some elements such as $f(x) = \frac{1}{x}$ in $x = 0$, you would want to put a double bar to signify that the value is undefined.

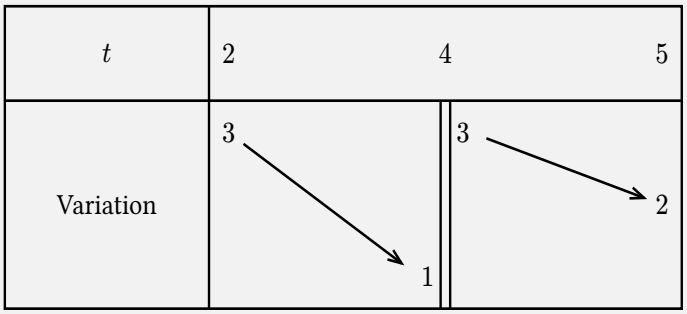
⋆ For each value of the domain exept the first and last :

The array must have this form `(pos1, pos2, "||", element1, element2)` where :

- `pos1` and `pos2` are `top`, `center` or `bottom`
- `"||"` is thre to specify that the value is undefined
- `element1` et 2 are of `contents` type where `element1` is the element before the bar and `élément2` after

Example :

```
#tabvar(
  variable: $t$,
  label: (
    ([Variation], "v"),
  ),
  domain: ($ 2 $, $4$, $5$),
  contents: (
    (
      (top, $3$),
      (bottom, top, "||", $1$, $3$),
      (center, $2$),
    ),
  ),
)
```

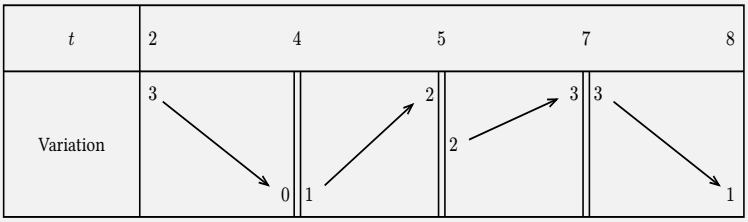


In the case where `pos1` et `pos2` are identical, you can put only one of them, and the same is possible for `element1` et `2`.

Example :

Instead of `(top, top, "||", $0$, $0$)`, you can write `(top, "||", $0$)`.

```
#tabvar(
  variable: $t$,
  label: (
    ([Variation], "v"),
  ),
  domain: ($ 2 $, $4$, $5$, $7$, $8$),
  contents: (
    (
      (top, $3$),
      (bottom, "||", $0$, $1$),
      (top, center, "||", $2$),
      (top, "||", $3$),
      (bottom, $1$),
    ),
  ),
)
```

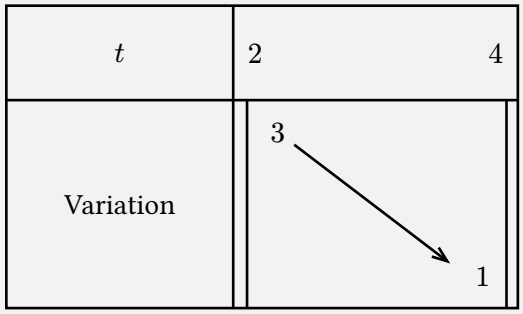


⋆ For the beginning and the end

As there is only one element here, the array is in the compressed notation seen previously, i.e. : `(pos, "||", element)`.

Example :

```
#tabvar(
  variable: $t$,
  label: (
    ([Variation], "v"),
  ),
  domain: ($ 2 $, $4$),
  contents: (
    (
      (top, $3$),
      (bottom, "||", $1$),
    ),
  ),
)
```

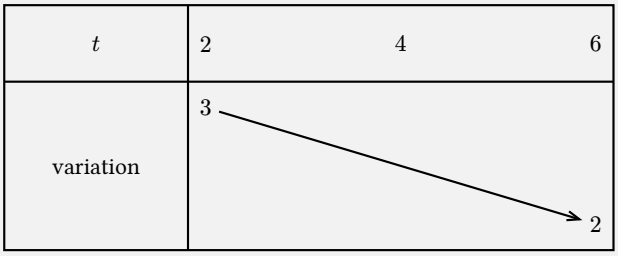


3.2.3 - Ignoring a value

When you use several function in the same table of signs, you might want to ignore some values of the domain, in order to do so, as with the sub-tables of signs, you only need to put an empty array « `()` ».

Example :

```
#tabvar(
  variable: $t$,
  label: (([variation], "v"),),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      (top, $3$),
      (),
      (bottom, $2$),
    ),
  ),
)
```



3.3 - Hatching for an undefined interval

Comparing to sub-tables of signs, elements here are for each value of the domain, and not only intervals.
Therefore, to signify that an interval is undefined, we will use 4 tags, including two of « openning » and two of « closing ».

- The « openning » tags are : `"h"` et `"|h"`, the second tag precise that the function is not defined for this value.
- The « closing » tags are : `"H"` et `"H|"`, the second tag precise that the function is not defined for this value.

Like this you will only need to put this tag between the alinment and the value of the function, e.g. `(top, [tag], $3)`.

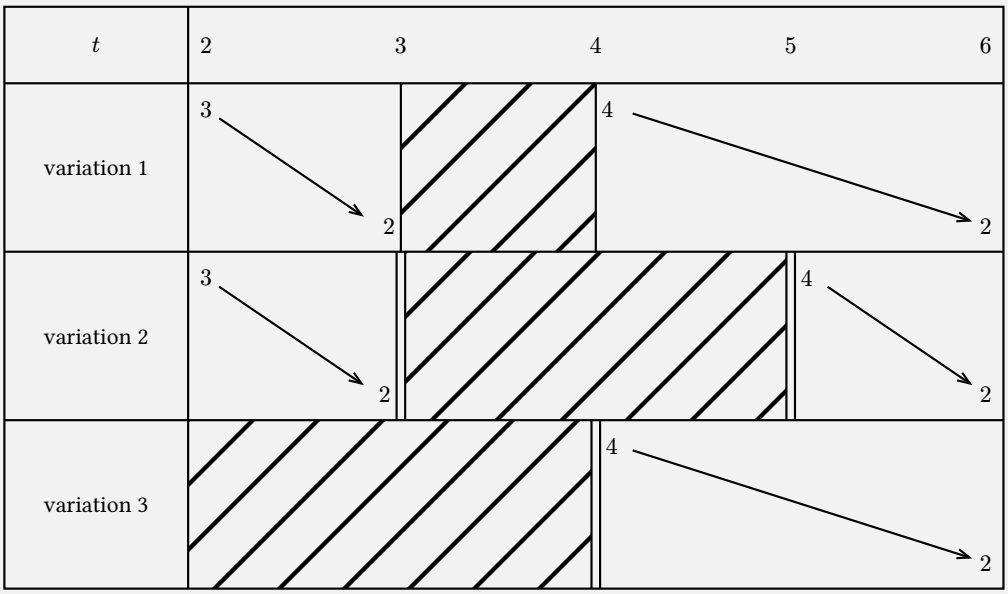
Additionally to extend the hatching to more than one interval,you only need to put empty arrays between two elements containing tags of openning and closing.

Warning :

- The « `|h` » and « `H|` » tags are respectively not compatible with the first and last element.
- And beware not to put non empty elements between two tags, it breacks the table.

Example :

```
#tabvar(
  variable: $t$,
  label: (
    ([variation 1], "v"),
    ([variation 2], "v"),
    ([variation 3], "v"),
  ),
  domain: ($2$, $3$, $4$, $5$, $6$),
  contents: (
    (
      (top, $3$),
      (bottom, "h", $2$),
      (top, "H", $4$),
      (),
      (bottom, $2$),
    ),
    (
      (top, $3$),
      (bottom, "|h", $2$),
      (),
      (top, "H|", $4$),
      (bottom, $2$),
    ),
    (
      (bottom, "|h", $2$),
      (),
      (top, "H|", $4$),
      (),
      (bottom, $2$),
    ),
  ),
)
```



4 - Resizing the table

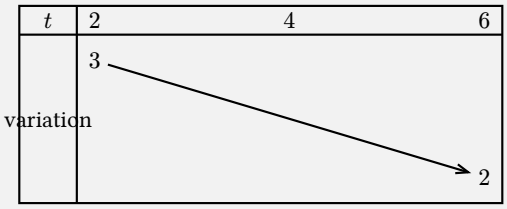
4.1 - First line and column

As seen in section 2, there are two parameters doing exactly what is discussed here, i.e. modifying first line and column.

Those two parameters take a `lenght` type, nervertheless they have to be positive !

Example :

```
#tabvar(
  variable: $t$,
  label: (([variation], "v"),),
  domain: ($2$, $4$, $6$),
  first-column-width: 1cm,
  first-line-height: 5mm,
  contents: (
    (
      (top, $3$),
      (),
      (bottom, $2$),
    ),
  ),
)
```



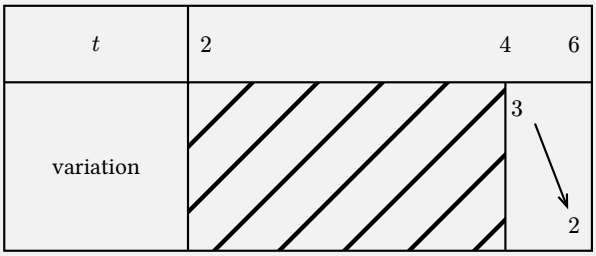
N.B.: If those two parameters are not filled, then the height and width will match the size of the contained text,
however, if said text is too small, the first column will be 30mm large and the first line 12mm high.

4.2 - Resizing the spacing between elements

To modify the spacing between the elements of the domain, substitute the element before the spacing by a couple « `(content, lenght)` », where `content` is the element of the domain at that point, and `lenght` the distance between said element and the next.
As you may have understood, the last element cannot be substituted by said couple.

Example :

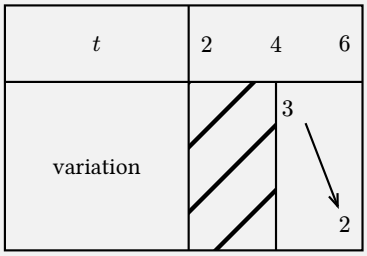
```
#tabvar(
  variable: $t$,
  label: (([variation], "v"),),
  domain: (($2$, 5cm), ($4$, 1cm), $6$),
  contents: (
    (
      "h",
      (top, "H", $3$),
      (bottom, $2$),
    ),
  ),
)
```



If you want to change all the spacing the same way, you only need to use the `element-distance` parameter.

Example :

```
#tabvar(
  variable: $t$,
  label: (([variation], "v"),),
  domain: ($2$, $4$, $6$),
  element-distance: 1cm,
  contents: (
    (
      "h",
      (top, "H", $3$),
      (bottom, $2$),
    ),
  ),
)
```

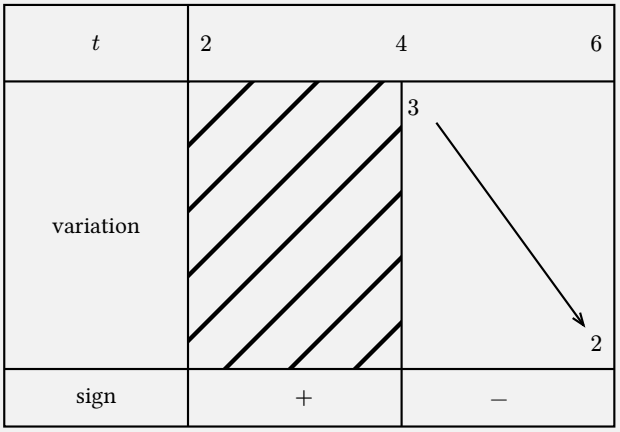


4.3 - Resizing the height of sub-tables

To chage this height, add in the label, in the array conresponding to the sub-table, between content and sign `"s"` or variation `"v"` tag, the height you want.
Know that the default height is 13,5 mm minimum .

Example :

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], 5cm, "v"),
    ([sign], 10mm, "s"),
  ),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      "h",
      (top, "H", $3$),
      (bottom, $2$),
    ),
    ($+$, $-$),
  ),
)
```



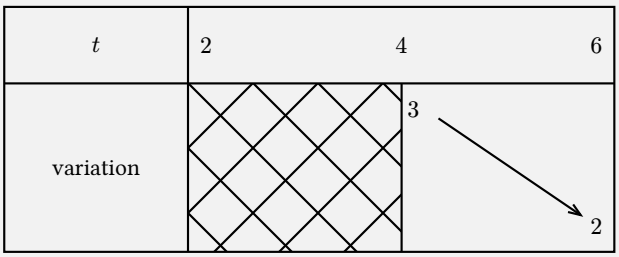
5 - Customing the hatching

In order to do so you only need to put an object of `tiling` type to the `hatching-style` parameter.

Example :

```
  #tabvar(
    #let exemple = tiling(size: (30pt, 30pt))[
    #place(line(start: (0%, 0%), end: (100%,
100%)))
    #place(line(start: (0%, 100%), end: (100%,
0%)))
  ]

  #tabvar(
    variable: $t$,
    label: (
      ([variation], "v"),
    ),
    domain: ($2$, $4$, $6$),
    hatching-style: exemple,
    contents: (
      (
        "h",
        (top, "H", $3$),
        (bottom, $2$),
      ),
    ),
  )
```



Additionally the package comes with its own pre-defined hatchings, done by Alkion (huge thank to him), here is the presentation :

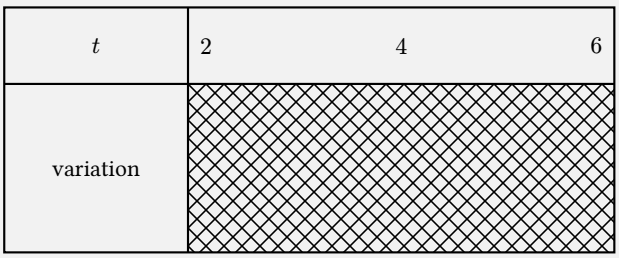
⋆ `grille`

Definition :

```
  #let grille = tiling(size: (8pt, 8pt))[
    #place(line(start: (0%, 0%), end: (100%, 100%), stroke: .7pt))
    #place(line(start: (0%, 100%), end: (100%, 0%), stroke: .7pt))
  ]
```

Example :

```
  #tabvar(
    variable: $t$,
    label: (
      ([variation], "v"),
    ),
    domain: ($2$, $4$, $6$),
    hatching-style: grille,
    contents: (
      (
        "h",
        (),
        (top, "H", $3$),
      ),
    ),
  )
```



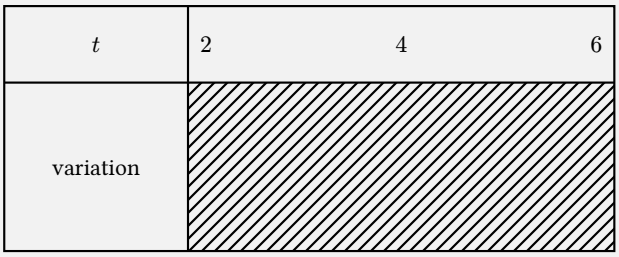
⋆ `nelines`

Definition :

```
#let nelines = tiling(size: (6pt, 6pt))[
  #place(line(start: (100%, 0%), end: (0%, 100%)))
  #place(line(start: (100%, -100%), end: (-100%, 100%)))
  #place(line(start: (200%, 0%), end: (0%, 200%)))
]
```

Example:

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
  ),
  domain: ($2$, $4$, $6$),
  hatching-style: nelines,
  contents: (
    (
      "h",
      (),
      (top, "H", $3$),
    ),
  ),
)
```



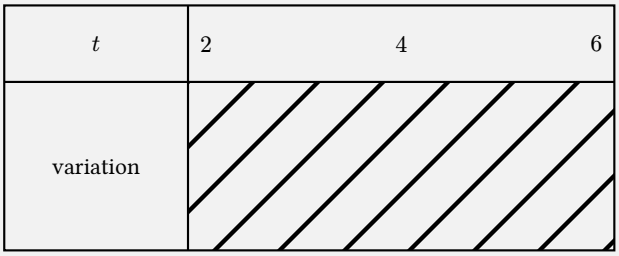
⋆ bignelines

Definition:

```
#let bignelines = tiling(size: (30pt, 30pt))[
  #place(line(start: (100%, 0%), end: (0%, 100%), stroke: 2pt))
  #place(line(start: (100%, -100%), end: (-100%, 100%), stroke: 2pt))
  #place(line(start: (200%, 0%), end: (0%, 200%), stroke: 2pt))
]
```

Example:

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
  ),
  domain: ($2$, $4$, $6$),
  hatching-style: bignelines,
  contents: (
    (
      "h",
      (),
      (top, "H", $3$),
    ),
  ),
)
```



⋆ nwlines

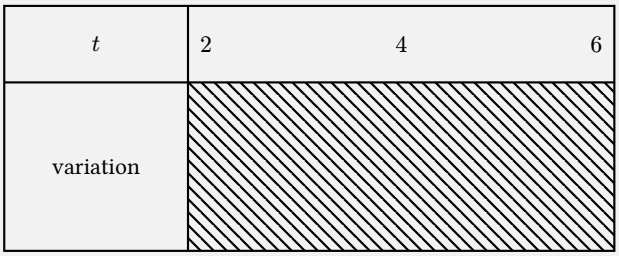
Definition:

```
#let nwlines = tiling(size: (6pt, 6pt))[
  #place(line(start: (0%, 0%), angle: 45deg))
  #place(line(start: (-100%, 0%), angle: 45deg))
```

```
    #place(line(start: (100%, 0%), end: (200%, 100%)))
    #place(line(start: (100%, -100%), angle: -135deg))
  ]
```

Example :

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
  ),
  domain: ($2$, $4$, $6$),
  hatching-style: nwlines,
  contents: (
    (
      "h",
      (),
      (top, "H", $3$),
    ),
  ),
)
```



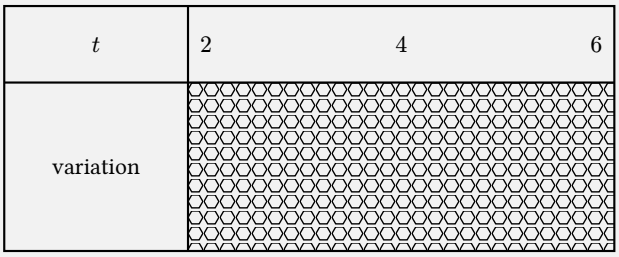
⋆ `hatch`

Definition :

```
#let hatch = tiling(size: (7pt, 7pt))[
  #place(polygon.regular(vertices: 6, size: 6.5pt, stroke: .6pt))
]
```

Example :

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
  ),
  domain: ($2$, $4$, $6$),
  hatching-style: hatch,
  contents: (
    (
      "h",
      (),
      (top, "H", $3$),
    ),
  ),
)
```



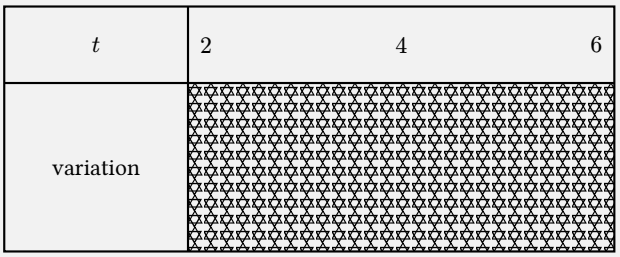
⋆ `etoile`

Definition :

```
#let etoile = tiling(size: (7pt, 7pt))[
  #place(rotate(180deg, origin: center + horizon)[#polygon.regular(vertices: 3,
size: 6.5pt, stroke: .6pt)])
  #place(polygon.regular(vertices: 3, size: 7pt, stroke: .5pt))
]
```

Example :

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
  ),
  domain: ($2$, $4$, $6$),
  hatching-style: etoile,
  contents: (
    (
      "h",
      (),
      (top, "H", $3$),
    ),
  ),
)
```



6 - Adding values into sub-tables of variations

Indeed it is possible to add values into the sub-tables of variations , without expanding the domain. This can be useful to those who would want to show on the table a precise value taken by the function found throught intermediate values theorem.

now come the use of the `values` argument, indeed you will put into `values` as many values as you will want to add.
The element you add must be like : `("arrowxy", content1, content2)`, where :

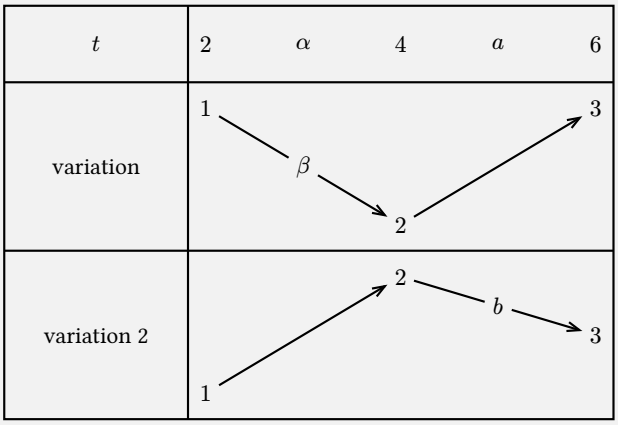
- `x` and `y` in `"arrowxy` are the coordinates of the arrow in which you want to add a value, those coordinates start from the top left corner with `x = 0, y = 0`

y	0	1	2	3	4
`x = 0`	1 arrow00 1		arrow01 1		arrow02 1 arrow03 1
`x = 1`	1 arrow10 1		arrow11 1		arrow12 1 arrow13 1
`x = 2`	1 arrow20 1		arrow21 1		arrow22 1 arrow23 1

- `content1` the content placed inside the domain
- `content2` the content place on the arrow

Example :

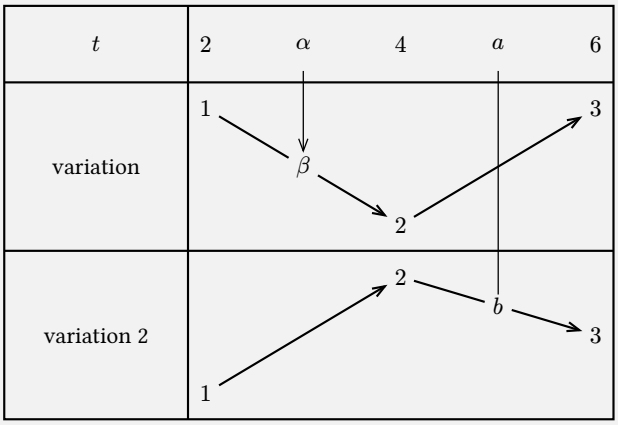
```
#tabvar(
  fill-color: luma(95%),
  variable: $t$,
  label: (
    ([variation], "v"),
    ([variation 2], "v"),
  ),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      (top, $1$),
      (bottom, $2$),
      (top, $3$),
    ),
    (
      (bottom, $1$),
      (top, $2$),
      (center, $3$),
    ),
  ),
  values: (
    ("arrow00", $alpha$, $beta$),
    ("arrow11", $a$, $b$),
  ),
)
```



Moreover, it is possible to add an arrow or a line joining the value inside the domain and the one on the arrow.

Example :

```
#tabvar(
  fill-color: luma(95%),
  variable: $t$,
  label: (
    ([variation], "v"),
    ([variation 2], "v"),
  ),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      (top, $1$),
      (bottom, $2$),
      (top, $3$),
    ),
    (
      (bottom, $1$),
      (top, $2$),
      (center, $3$),
    ),
  ),
  values: (
    ("arrow00", $alpha$, $beta$, "f"),
    ("arrow11", $a$, $b$, "l"),
  ),
)
```



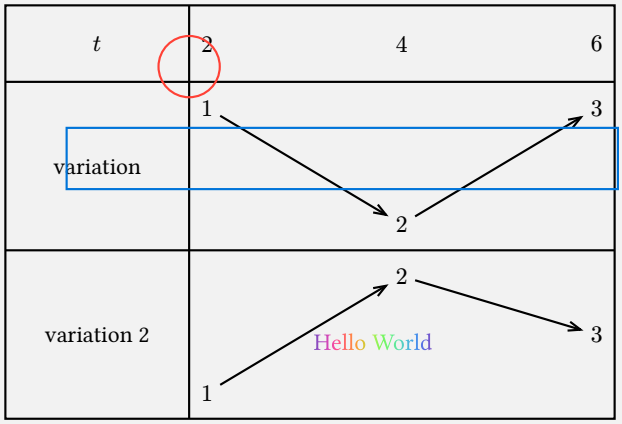
7 - Add what you want with `add`

It is possible to add as many alement as you want (as long as cetz support it) to your tables, in order to do so, you only have to add those elements in the `add` parameter.

Warning : To add element specific to Cetz, such as `content, rect, etc` you must import them into your Cetz file.

Example :

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
    ([variation 2], "v"),
  ),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      (top, $1$),
      (bottom, $2$),
      (top, $3$),
    ),
    (
      (bottom, $1$),
      (top, $2$),
      (center, $3$),
    ),
  ),
  add: {
    cetz.draw.circle((3, -1), stroke: red, radius: 5mm)
    cetz.draw.content((6, -5.5), text(fill: gradient.linear(..color.map.rainbow))
[Hello World])
    cetz.draw.rect((1, -2), (10, -3), stroke: blue)
  },
)
```



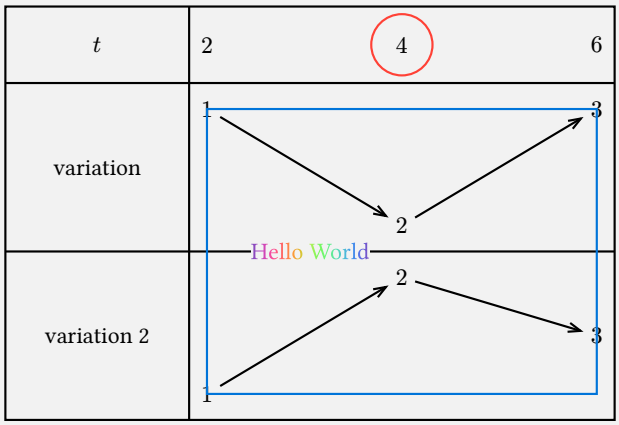
To simplify the process each element of the table has a « name » that permits, throught Cetz coordinate system, to attach the added elements to the elements already there.
Here is a table that sum up the names :

elements	name	precision
the variable	`var`	
domain	`domainx`	`x` represents the x-th element of the domain
label	`labely`	`y` reprsents the y-th label
line between subtables	`line-betwen-table-nby`	`y` represents the y-th line note : line 0 separate the domain from the rest
cadre	`cadre`	always possible to use even with `nocadre` at `true`
line between labels and subtables	`line-separating-labels-tables`	
line going through the domain, centered	`line-centred-domain`	invisible line

arrows inside the subtables of variations	`arrowxy`	are exactly the one in section 6 see for more precisions
elements in a subtable of variations	`variationxy`	x and y are the element's coordinates
elements in a subtable of signs	`signxy`	x and y are the coordinates of the bar. works the same way as for `arrowxy`.
hatching	`hatchingxy`	x and y are the hatching coordinates
element of the domain, for an added value	`depart_valuesx`	x is the x-th added element
element inside the subtable of variations, for an added value	`fin_valuesx`	x is the x-th added element

1er Example :

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
    ([variation 2], "v"),
  ),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      (top, $1$),
      (bottom, $2$),
      (top, $3$),
    ),
    (
      (bottom, $1$),
      (top, $2$),
      (center, $3$),
    ),
  ),
  add: {
    import cetz.draw: *
    circle("domain1", stroke: red, radius: 5mm)
    content(
      "line-betwen-table-nb1",
      text(fill: gradient.linear(..color.map.rainbow))[Hello World],
      frame: "rect",
      fill: luma(95%),
      stroke: none,
    )
    rect("variation02", "variation10", stroke: blue)
  },
)
```



2ième Exemple :

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
    ([variation 2], "v"),
  ),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      (top, $1$),
      (bottom, $2$),
      (top, $3$),
    ),
    (
      (bottom, $1$),
      (top, $2$),
      (center, $3$),
    ),
  ),
  add: {
    import cetz.draw: *
    polygon("line-centred-domain.82%", 5, stroke: red, radius: 5mm)
    line("var", "label1", stroke: blue)
    image("")
  },
)
```

